XE

XELOR · PRODUCT DEVELOPMENT

Product Development Master Plan

The Product Development Master Guide applied, section by section, to what XELOR actually is today — what is built and proven, what is partial, and what has not been started.

Assessed against the running Phase 1 and Phase 2 stacks, the repository, the test suites and the CI pipeline

12 August 2026

How to read this document

The attached Master Guide is a checklist for planning a complex platform. This document is that checklist **answered** for XELOR: every one of its thirteen sections, with the real state of the product against each.

Three rules were followed throughout, because a plan that flatters itself is worth nothing:

- **Every claim is evidence-backed.** Counts come from the repository and the running stacks, not from memory. Where a number is quoted it can be re-derived with the command in the Evidence appendix.
- **Gaps are stated as gaps.** An item the guide asks for that XELOR has not built is marked GAP , not softened into "planned".
- **Not-applicable is a real answer.** Some of the guide targets consumer-scale SaaS. Where an item does not apply to an India-first manufacturing ERP at MVP stage, it is marked N/A with the reason.

The product, in one paragraph

XELOR is an India-first, AI-native manufacturing ERP for MSMEs, built as a boundary-enforced modular monolith. It ships in two layers. **Phase 1** is the ERP and system of record — Sales, Inventory, Purchase, Production, Engineering, Planning, Quality, Accounts. **Phase 2** is an intelligence and automation layer that sits on top: it reads Phase 1, works out what should happen, explains its reasoning, asks a human for authority where the decision commits money, and then writes real documents back through Phase 1's own service ports. Both layers run today.

988	450	251	24
unit & service tests passing	API endpoints across 33 controllers	tables over 96 migrations	installed product modules

> **The headline verdict.** XELOR is unusually strong on the guide's sections 2, 4, 6 and 8 — architecture, backend, security and testing. It is materially incomplete on sections 5, 9 and 10 — infrastructure/DevOps, observability, and release/incident operations — which is the normal shape for a product at investor-MVP stage but is exactly what must close before a paying pilot runs unattended.

1. Strategy & Requirements

Guide asks for	State	What exists in XELOR
Vision / problem statement	LIVE	"Turn disconnected factory data into trusted operational decisions in minutes — with evidence, under human control." Expressed as Connect → Retrieve → Reason → Act.
Target personas	LIVE	Indian MSME manufacturers. Two product forms: factories with no ERP (buy Phase 1), factories already running SAP/Tally/Odoo (buy Phase 2 alone). Five seeded personas drive the RBAC proof suite.
Market / competitor research	LIVE	Competitive set mapped across three axes — coordinated agents, sits-on-top, India-first MSME — with the gap being the overlap. Documented in the investor deck.
Success metrics (KPIs/OKRs)	PARTIAL	Value metrics are quantified (recovery of ~2% of turnover; 1,262 engineer-hours; ~₹42 L/yr on a ₹20 cr plant). **Product** KPIs — activation, time-to-first-value, retention — are not instrumented.
Pricing / monetisation	GAP	Explicitly an open item in the binding decisions register (§6c). No price band is committed, which is why the ROI analysis states a recovery ceiling rather than a payback period.
Roadmap with phased milestones	LIVE	Phase 1 frozen and tagged; Phase 2 built on top with its own database and ports. Both run side by side, which is itself the roadmap made testable.
Go-to-market plan	LIVE	Peenya (Bengaluru) industrial cluster first — ~13,000 units in one geography — then Coimbatore. 3–4 pilots identified.
Functional requirements	LIVE	Twenty-section module blueprints per department, each with acceptance criteria.
Non-functional requirements	PARTIAL	Security and correctness NFRs are specified and enforced. **Performance** NFRs exist as one rule (RLS overhead >15–20% triggers design review) but there are no latency, throughput or concurrency targets.
Acceptance criteria per feature	LIVE	A 99-check investor acceptance matrix runs in CI against a database rebuilt from empty.
Wireframes / design system	LIVE	Design system is enforced in code: every colour is a CSS custom property with a light and dark definition; a literal hex in a screen is treated as a defect.

> **Priority action.** Pricing (§6c) is the highest-value open item in this section. It blocks the payback narrative that the ROI figures are built to support, and every pilot conversation eventually asks for it.

2. Architecture & System Design

The core decision, and why

Boundary-enforced modular monolith — not microservices. For a five-engineer team shipping an ERP whose ledger writes must be transactional, microservices would buy independent deployability at the cost of distributed transactions across Sales, Inventory and Accounts. The boundaries are real but enforced in CI rather than over a network: no cross-module imports, no hard foreign key across a module edge, and cross-module access only through a declared service port or an outbox event. `module-check` fails the build on violation — it currently reports **24 modules, 169 nav permissions, no cross-module imports**.

The two-layer architecture, which is the product's actual thesis

Phase 2 may reach Phase 1 **only** through ports. This is enforced by construction: the fulfilment module does not import the Purchase or Production modules at all — it resolves `PURCHASE_ORDER_WRITER` and `PRODUCTION_ORDER_WRITER` tokens and can therefore reach an interface but not a service class. The intelligence engine sits behind a single `IntelligenceEngine` interface with a deterministic implementation, so a real model can be substituted later without rewriting the orchestration.

Guide asks for	State	What exists
Monolith vs microservices decision	LIVE	Modular monolith, decided and enforced in CI. Rejected alternatives are recorded.
Tech stack	LIVE	NestJS 11 / Node 22 · Next.js 15 / React 19 · PostgreSQL 17 · Drizzle ORM · Valkey + BullMQ · Keycloak 26 · pgvector · Gotenberg · AWS ap-south-1.
API style & versioning	LIVE	REST under a global `/api/v1` prefix. Events are versioned separately as `module.entity.verb.v1`.
Data model & schema	LIVE	251 tables, 96 hand-written SQL migrations. UUIDv7 keys, `tenant_id` on every tenant-scoped table, money as `NUMERIC(18,2)`, soft delete, no hard DELETE on financial or statutory rows.
Third-party integration points	PARTIAL	Spreadsheet import is genuinely built and writes through the domain APIs. Eight further connectors (SAP, Tally, Odoo, Dynamics 365, MES/SCADA, REST, Database, Excel) are catalogued and **honestly marked not connected** — the type forbids flipping one to connected without an implementation.
Caching strategy	PARTIAL	Valkey is deployed and used by BullMQ. It is **not** used as a read cache; notably the permission guard queries the database on every guarded request with no cache.
Horizontal scaling	PARTIAL	Services are stateless and the tenant context is per-request, so horizontal scaling is architecturally available. It has never been exercised — no load balancer, no multi-instance run.
DB scaling (replicas, sharding)	GAP	Single primary. No read replica, no partitioning strategy for the append-only ledger tables that will grow fastest.
Rate limiting / throttling	GAP	Not implemented at the application or gateway layer.
Performance targets	GAP	No documented latency, throughput or concurrency targets, and therefore nothing to test against.

3. Frontend

Guide asks for	State	What exists
Framework & state management	LIVE	Next.js 15 App Router, React 19. State is deliberately hand-rolled `useState` plus React context — no Redux, no Zustand.
Component library / design system	LIVE	A shared "spine": PageHeader, DataTable, StatusBadge, Modal, Can, plus a single pipeline/layer visual language that draws Phase 1 and Phase 2 differently on every screen.
CSS architecture	LIVE	Every colour is a CSS custom property defined for light and dark. Ink tokens and fill tokens are separate, because a colour legible as text on a surface is not legible as white-on-fill.
Build tooling / code splitting	LIVE	Next build with per-route chunking; module screens are dynamically imported from each module's manifest.
Accessibility	PARTIAL	Keyboard focus is visible, controls are native elements, decorative icons are `aria-hidden`, reduced-motion respected. **No WCAG audit has been run** and there is no automated a11y gate.
Responsive breakpoints	PARTIAL	Screens use container queries and reflow correctly on a laptop. **Mobile and offline-first is the single biggest declared gap** in the platform — and a factory floor is a mobile, intermittently-connected environment.
Core Web Vitals	GAP	Not measured. No LCP/CLS budget, no Lighthouse gate in CI.
Testing: unit / E2E	LIVE	58 web unit tests; 12 Playwright specs covering the demo, agent OS, RBAC personas and the sign-in theme.
Visual regression	GAP	None. Every visual defect found this cycle — a collapsed table column, an invisible button, a low-contrast fill — was found by a human looking at a screenshot.

> **Priority action.** Mobile/offline-first is the biggest gap in the whole document, and it is not merely a frontend concern: it determines whether a supervisor can record output at the machine or must walk to an office PC. It is already recorded as a blocking gate in the binding decisions register (§6a).

4. Backend

Guide asks for	State	What exists
Controllers / services / repositories	LIVE	33 controllers, 450 endpoints, 223 API source files. Domain logic lives in services; controllers validate and delegate.
Standardised error format	LIVE	One canonical envelope: `{ error: { code, message, details[], traceId } }`. Field-level `details` survive all the way into the spreadsheet importer's per-row failure messages.
Input validation	LIVE	Zod `safeParse` per endpoint. Deliberately not a global pipe — the trade-off is noted below.
Pagination	LIVE	Cursor pagination only, capped at 100 rows. Offset pagination is prohibited.
Idempotency	LIVE	`Idempotency-Key` required on every mutating endpoint, backed by a ledger. Phase 2 derives deterministic keys from mission + plan version + vendor, so a retried step cannot raise a second purchase order.
AuthN / AuthZ	LIVE	Keycloak 26 OIDC; 165 registered permissions with RBAC/ABAC. A dedicated Playwright suite proves five personas see only their own departments and that the API refuses on the same boundary.
Transactions	LIVE	Ledger-critical writes stay synchronous in one transaction and never travel on the bus. Cross-module document creation is deliberately a separate transaction via a port — documented, because nesting them would deadlock the pool.
Job queues	PARTIAL	Valkey + BullMQ are deployed. Phase 2 missions advance only when called — there is no background worker driving them.
Eventing	PARTIAL	Transactional outbox exists (at-least-once + idempotent consumer = exactly-once effect). Phase 2 currently publishes one event; plan, approval and action events are not yet emitted.
File storage	PARTIAL	Import accepts base64 in a JSON body — a deliberate choice to inherit the existing authenticated pipeline rather than add an unguarded multipart route. The cost is a ~64 KB ceiling. No object-storage client is wired.
N+1 / indexing	LIVE	Composite indexes lead with `tenant_id`. Hot paths batch their reads — the procurement step loads the vendor and item masters in one query each rather than per line.

An honest trade-off worth restating. Validation is per-controller rather than a global pipe. The benefit is that each endpoint's contract is explicit and testable; the risk is that a new endpoint which forgets `safeParse` receives unvalidated input. That risk is currently mitigated by review alone. A lint rule asserting every `@Body()` is parsed would close it cheaply.

5. Infrastructure & DevOps

Guide asks for	State	What exists
Cloud provider & region	LIVE	AWS ap-south-1 (Mumbai), chosen for data residency under India's DPDP Act rather than latency.
Infrastructure as Code	PARTIAL	OpenTofu is the decided tool. Deployment today is Docker Compose for local and a hosted demo stack.
Secrets management	PARTIAL	`.env` is git-ignored and no secret is committed — verified by scanning every commit in this cycle; only variable *references* appear. There is no Vault or Secrets Manager integration and no rotation policy.
CI/CD pipeline	LIVE	GitHub Actions, two jobs. **verify**: build, lint (module boundaries), typecheck, 988 tests, migrate, then RLS/permission/naming/manifest proofs against a real PG17. **investor-demo**: rebuilds the world from an empty database and runs the 99-check matrix.
Branching strategy	PARTIAL	Trunk-based on `main`. No protected-branch rule, no required review — which is how two commits reached the public repository this cycle without an approval step.
Containers	LIVE	Docker Compose brings up Postgres, Valkey, Keycloak and Gotenberg. Production compose files exist.
Orchestration (Kubernetes)	N/A	Correctly out of scope. A modular monolith serving MSME tenants does not need Kubernetes before it needs paying customers.
Deployment strategy	GAP	No blue-green, canary or rolling strategy defined. Deployment is restart-in-place.
Rollback automation	GAP	None. Migrations are forward-only by design — the migrator refuses to run if an applied file's hash changed — but there is no rollback path and no down migrations.
Container vulnerability scanning	GAP	Not configured.

The most consequential finding in this document. Two commits were made and pushed to a **public** repository during this development cycle without a review or approval step. The content was audited afterwards and is clean — no `.env` is tracked, no literal credential appears, and the only password-shaped strings are variable references. But the control that should have prevented an unreviewed push to a public repository does not exist. **Enable branch protection on `main` with a required review before the next change lands.**

6. Security & Compliance

Guide asks for	State	What exists
Multi-tenant isolation	LIVE	**PostgreSQL FORCE ROW LEVEL SECURITY on 65 tables**, enforced for the schema owner too. A two-tenant leak probe runs in CI. This is the strongest control in the platform.
Password hashing / MFA	LIVE	Delegated to Keycloak 26 — credentials never reach application code.
Parameterised queries (SQLi)	LIVE	Drizzle ORM throughout; no string-built SQL in application paths.
Input validation / XSS	LIVE	Zod at every mutating endpoint; React escapes by default and no `dangerouslySetInnerHTML` is used in product screens.
Audit logging	LIVE	**Hash-chained** audit trail. Fifty-four tables carry an append-only trigger that fires for the schema owner as well, so a row-level undo is impossible by construction — which is what the MCA eight-year retention requirement actually demands.
Data residency & DPDP	LIVE	ap-south-1 only. Compliance facts (DPDP, CERT-In, GST including the 1 Aug 2026 Ship-to-GSTIN rule, payroll deemed wages) are recorded as binding decisions, and every statutory number is configuration rather than a constant in code.
Encryption in transit / at rest	PARTIAL	TLS at the edge in the hosted deployment. No documented KMS strategy or at-rest encryption policy.
Dependency scanning	PARTIAL	The spreadsheet library is deliberately pinned to the vendor's patched distribution rather than the vulnerable registry build — a considered decision, documented in code. But there is **no automated Dependabot/Snyk gate**.
SAST / DAST	GAP	Not configured.
Penetration testing	GAP	Never performed.
Security headers / rate limiting	GAP	No CSP, HSTS or X-Frame-Options set by the app; no API rate limiting.
Incident response plan	GAP	Not written. CERT-In imposes a six-hour incident reporting obligation in India — this gap has a regulatory deadline attached to it.

> **The public-demo bypass.** A static header authenticates as a demo administrator without a token. It is gated on a runtime check that the database contains *only* the two seeded demo tenants and refuses otherwise. That is a good control, and it must never front real customer data.

7. Data Management & Analytics

Guide asks for	State	What exists
Data collection / ingestion	LIVE	Named ingestion layer with nine connected Phase 1 sources, two honest stand-ins for facts Phase 1 holds no table for, and spreadsheet upload.
Normalisation	LIVE	Canonical internal models — Customer, Item, StockLine, Supplier, OrderLine — with explicit hand-written mappers. Deliberately concrete rather than a generic mapping engine.
ETL pipelines	PARTIAL	The import path is a real inspect → map → validate → commit pipeline with a persisted batch and per-row outcome, so a partial import is auditable and resumable. There is no scheduled or streaming ETL.
Data quality / governance	LIVE	Row-level validation with field-level messages; duplicate detection that explains itself (matching on GSTIN and reporting name similarity); effective-dated statutory masters.
Retention / archival	PARTIAL	Eight-year retention is a stated compliance fact and append-only triggers enforce immutability. No archival or partitioning mechanism exists for when those tables get large.
Product event tracking	GAP	No user-behaviour analytics, no funnels. The system records what the *business* did in exhaustive detail and nothing about what the *user* did.
BI / dashboards	PARTIAL	Each module declares its own signals and alerts for a department dashboard. There is no cross-tenant BI or warehouse.
ML models	N/A	Deliberately none. The planner is symbolic and deterministic — research-backed as *better* than an LLM for a closed manufacturing domain, because a model can hallucinate a part number and a planner cannot. The seam for a real model is a single interface.

8. Testing Strategy

821	104	58	12
platform unit tests	API tests	web tests	Playwright E2E specs

Layer	State	What exists
Unit	LIVE	988 tests across four packages, 84 test files, Node's built-in runner. All passing; one deliberate environment-dependent skip.
Integration	LIVE	DB-gated proofs run against a real PG17 in CI: RLS, permission coverage, naming conventions, module manifests, and a two-tenant leak probe.
E2E	LIVE	12 Playwright specs. Skips are declared preconditions, not silent disappearances — the two auth-mode-specific suites call `test.skip` with a reason.
Acceptance	LIVE	A 99-check matrix run against a world rebuilt from an **empty** database. This job is not redundant: a defect once passed every static check and all unit tests because the already-seeded database happened to hold the rows being asserted.
Invariant / stress	LIVE	A 288-scenario harness asserts invariants rather than outputs — change one fact, prove the plan changes — which is what distinguishes a deterministic engine from a script.
Performance / load	GAP	No k6, JMeter or Locust. Nothing has ever been load tested. This is the largest single gap in the testing section.
Security testing	GAP	No OWASP scan, no ZAP/Burp.
UAT	PARTIAL	Pilots identified; no structured UAT has run with a real plant.
Coverage enforcement	GAP	Tests run in CI but no coverage threshold is enforced, so coverage can silently fall.

> **What is genuinely excellent here.** The rebuild-from-empty acceptance job and the invariant harness are both more rigorous than most products carry at this stage, and both exist because a real defect got through a weaker check.

9. Monitoring & Observability

Guide asks for	State	What exists
APM	GAP	OpenTelemetry, Grafana and Sentry are the *decided* stack. None is wired into the running application.
Centralised logging	GAP	Logs go to a local file per process. No aggregation, no structured JSON contract, no retention policy.
Error tracking	PARTIAL	Every error carries a `traceId` in the canonical envelope — the hard part is already done. Nothing collects them.
Uptime monitoring / status page	PARTIAL	A platform-health module runs five probes and a scheduled check. There is no external uptime monitor and no public status page.
Distributed tracing	N/A	Correctly unnecessary for a modular monolith. The `traceId` already correlates a request end to end.
Alerting	PARTIAL	Business alerts are strong and deliberately rule-based — "an alert wrong in the reassuring direction is worse than no alert at all". Infrastructure alerting (error rate, latency, disk, memory) does not exist.
On-call rotation	GAP	None. No PagerDuty/Opsgenie, no escalation policy.
Runbooks	PARTIAL	Operational traps are documented well in-repo — the stale-build trap, the seeder auth trap, the demo-reset semantics. These are runbooks in substance but not organised as incident procedures.

This is the weakest section of the entire assessment. The product can currently tell a factory in forensic detail what happened to a purchase order, and cannot tell its own operators that the API is failing. Before any pilot runs unattended, three things must exist: Sentry wired for errors, an external uptime check, and one alert that reaches a human.

10. Release & Operations

Guide asks for	State	What exists
Versioning	PARTIAL	API is versioned; events are versioned. The product itself carries no semantic version and releases are not tagged consistently.
Release notes	PARTIAL	Commit messages are unusually descriptive and explain intent. There is no changelog a customer could read.
Feature flags	PARTIAL	Environment flags exist (public-demo mode, autonomy tiers). No runtime flag service for gradual rollout.
Pre-deploy checklist	PARTIAL	CI enforces build, lint, typecheck, tests and the DB proofs. There is no explicit human gate or documented go/no-go.
Hotfix process	GAP	Undefined.
Severity levels / incident commander	GAP	Undefined.
Postmortems	PARTIAL	Not a formal practice — but the codebase does something rarer and arguably better: the *reason* a defect happened is written into the comment above the fix, so the lesson lives where the next engineer will read it.
Backups & tested restore	GAP	No automated backup, and no restore has ever been tested.
RTO / RPO	GAP	Not defined.
Failover / geographic redundancy	GAP	None. Single region, single primary.

Untested backups are the classic way to lose a customer permanently. For a system of record holding eight years of statutory data under an append-only guarantee, an automated backup with a *proven* restore is not optional past the first pilot. It is the cheapest item on this page and among the most consequential.

11. Team, Process & Governance

Guide asks for	State	What exists
Team size vs stage	LIVE	~5 engineers at MVP stage — within the guide's 5–8 band, though carrying an unusually large surface for that size.
Architecture Decision Records	LIVE	A binding platform-decisions register that **wins on conflict** with any module blueprint, with a named reviewer for divergence. Rejected alternatives are recorded, and the stack survived an adversarial review with none of fourteen recommendations overturned.
Ownership model	LIVE	Seven departments own defined systems of record, each with its own blueprint. Ownership is explicit rather than assumed.
Code review workflow	GAP	No enforced review. Directly demonstrated this cycle: two commits reached a public repository unreviewed.
Agile ceremony structure	PARTIAL	Phased milestones exist; no formal sprint cadence, standups or retros documented.
Risk register with owners	PARTIAL	Nine open items are enumerated with an accountable department. They are not scored by probability and impact, and have no dates.
Technical debt tracking	PARTIAL	Debt is unusually visible — written in comments at the point of compromise, with the reasoning. It is not aggregated anywhere a planner could prioritise it.

Risk register — scored

Risk	Probability	Impact	Rating	Mitigation
No backup / untested restore	Medium	High	CRITICAL	Automated nightly backup + a restore rehearsal before the first pilot.
No observability in production	High	High	CRITICAL	Wire Sentry; add an external uptime check and one paging alert.
Unreviewed pushes to a public repo	High	Medium	CRITICAL	Branch protection on `main` with a required review. Cost: minutes.
Mobile/offline absent on a factory floor	High	High	CRITICAL	Already a declared blocking gate. Decide the strategy before the UX freeze.
Never load tested	Medium	High	CRITICAL	Define latency/throughput targets, then a k6 baseline against the RLS query paths.
No incident response plan (CERT-In: 6 hours)	Low	High	MITIGATE	Write the plan; it is a regulatory obligation in India, not a nice-to-have.
Pricing undecided	High	Medium	MITIGATE	Blocks the payback story the ROI work supports.
Permission guard hits the DB per request	High	Low	TRACK	Cache in Valkey when load testing shows it matters — not before.

12. Documentation

Guide asks for	State	What exists
Architecture documentation	LIVE	Module blueprints per department on a common 20-section template, plus a rendered report set covering architecture, stack, agentic strategy and the Phase 1/Phase 2 split.
ADRs	LIVE	The binding decisions register, with a divergence process.
API documentation	GAP	**No OpenAPI/Swagger specification** for 450 endpoints. This is the clearest documentation gap and it directly obstructs any integration partner.
Schema documentation	PARTIAL	Migrations are hand-written and heavily commented; the Drizzle schema is typed. No generated ER diagram.
Runbooks	PARTIAL	Environment traps are documented candidly, including ones that cost real time. Not organised as incident procedures.
User-facing docs	PARTIAL	A presenter walkthrough, a capability-gaps note, and in-product "About this screen" descriptions that CI requires on every visible nav entry. No end-user manual.
Onboarding docs	LIVE	Environment notes are unusually honest — including a section correcting the file's own earlier claims after they were measured and found wrong.

A documentation practice worth keeping. This codebase writes the *why* above non-obvious decisions, including what was measured and what broke. That is why an outsider can reconstruct the reasoning behind a trade-off months later — and it is rare.

13. Launch Readiness

The guide's launch checklist, answered honestly for a first paying pilot — not for an investor demo, which XELOR is already ready for.

Checklist item	State	Note
Features complete for the pilot scope	READY	Order to purchase order to work order runs end to end and creates real documents.
Critical bugs fixed	READY	988 tests green; the defects found this cycle were fixed and covered by regression tests.
Security audit passed	NOT READY	No external audit or penetration test has been performed.
Load testing completed	NOT READY	Never run.
UAT completed	NOT READY	No structured UAT with a real plant.
Documentation finalised	PARTIAL	Strong internally; no OpenAPI and no end-user manual.
DB migrations tested	READY	CI rebuilds from empty on every run — the strongest possible migration test.
Backups verified	NOT READY	No backup exists to verify.
Disaster recovery tested	NOT READY	No RTO/RPO defined.
Status page ready	NOT READY	Internal probes only.
Analytics tracking verified	NOT READY	No product analytics.
Monitoring watched live	NOT READY	Nothing to watch.
Rollback plan ready	NOT READY	Forward-only migrations, no rollback path.

The sequence that closes the gap

#	Action	Effort	Why this order
1	Branch protection + required review on `main`	Minutes	Costs almost nothing and closes a control gap that has already been exercised once.
2	Automated backup with a rehearsed restore	1–2 days	The only item on this list whose absence can lose a customer permanently.
3	Sentry + uptime check + one paging alert	2–3 days	You cannot operate a pilot you cannot see. `traceId` already exists, so this is wiring, not design.
4	Decide the mobile/offline strategy	Design first	Already a declared blocking gate; it constrains UX decisions that are cheaper to make before the freeze than after.
5	Performance targets, then a k6 baseline	3–5 days	Targets must come first — a load test without a target produces numbers nobody can act on. Aim it at the RLS query paths.
6	Generate OpenAPI from the existing zod schemas	2–3 days	The contracts are already declared per endpoint; this is largely mechanical and unblocks integrators.
7	Incident response plan (CERT-In 6-hour duty)	1 day	A regulatory obligation in India, and cheap to write.
8	Dependency scanning + security headers	1 day	Both are configuration rather than engineering.
9	Structured UAT with the first Peenya pilot	Pilot-length	Everything above exists so that this can be survived.

The honest summary. XELOR has built the hard, expensive things first — tenant isolation, an immutable audit trail, enforced module boundaries, evidence-grounded decisions under a human approval gate, and a test suite that rebuilds the world from nothing to prove itself. What remains is largely *operational*: backups, observability, load characteristics and release discipline. That is a far better position to be in than the reverse, because the operational layer can be bought and wired in weeks, whereas retrofitting tenant isolation or an audit trail into a live product cannot.

Appendix · Evidence

Every figure in this document is re-derivable. Run these from the Phase 2 repository root.

Claim	Command
988 tests passing	`pnpm --filter @ind-core/platform test` · `@ind-core/api` · `@ind-core/web` · `@ind-core/db`
450 endpoints, 33 controllers	`grep -rhoE '@(Get|Post|Patch|Put|Delete|Sse)\(' --include='*.ts' apps/api/src | wc -l`
251 tables, 96 migrations	`ls packages/db/migrations/*.sql | wc -l`
FORCE RLS on 65 tables	`grep -rho 'FORCE ROW LEVEL SECURITY' packages/db/migrations/*.sql | wc -l`
24 modules, 169 nav permissions, no cross-module imports	`pnpm --filter @ind-core/web module-check`
165 registered permissions	`packages/platform/src/access/permission-registry.ts`
Real documents raised by Phase 2	`SELECT count(*) FROM purchase_order WHERE remarks LIKE '%MSN-%';`
Both layers running	Phase 1 `:3001` · Phase 2 `:3101`; Phase 1 returns 404 for `/fulfilment` and `/dataimport`

Prepared by applying the Product Development Master Guide to the XELOR platform as it stood on 12 August 2026. Assessments were made against the running stacks and the repository, not against intent. Items marked GAP are not criticisms of the work done — they are the list of what a first paying pilot will require.